HostelFix

Track: Backend Development **Stack:** TypeScript, Node.js, Express, MongoDB (Mongoose), Zod, Jest

Target user

Students living in a university hostel (who report faults) and the hostel maintenance staff and admins (who assign and resolve them).

Problem

Hostel faults like a leaking tap or a sparking socket are reported verbally or through scattered messages, so reports get lost, nobody knows who is handling what, and the same fault keeps returning to the same room with no record. HostelFix gives students one place to report a fault and staff one place to assign, track and close it, with a history of every status change.

Entities

- **User** (name, email, password hash, role: student, staff or admin)
- **Hostel** (name, location)
- **Room** (hostel, room number, floor)
- **Ticket** (title, description, category, priority, status, room, reported by, assigned to, recurring flag), with **comments** and **status history** embedded in the ticket

Core flow

Student creates a ticket (POST /tickets) -> staff lists open tickets (GET /tickets?status=open) -> staff assigns the ticket and moves its status (PATCH /tickets/:id/assign, PATCH /tickets/:id/status) -> student checks progress (GET /tickets/:id)

In scope

- Ticket CRUD with category, priority and a status lifecycle (open, assigned, in progress, resolved)
- A status history that records who changed a ticket's status and when
- Filtering, sorting and pagination on the ticket list
- A recurring-fault flag when the same room reports the same category again within 30 days
- Authentication and role-based access (student, staff, admin), plus comments on tickets (Week 2)

Out of scope

- Payments or billing
- Real-time chat or live notifications
- Photo and file uploads
- A front-end or mobile app
- Email or SMS notifications

Top 3 risks

- **Time and scope.** Three part-time weeks is tight, and extras like comments could eat the core. *Mitigation:* finish the ticket lifecycle and auth first, and treat anything else as optional.
- **Authorisation mistakes.** A student viewing or editing another student's ticket, or changing status, would be a serious bug. *Mitigation:* write ownership and role checks as middleware, and test the forbidden cases.
- **Free hosting and database setup.** A free host sleeps when idle and a wrong connection string can break the live API. *Mitigation:* deploy in Week 1, note the sleep behaviour in the README, and re-check the live URL before each submission.